

Bayesian Game Analysis on The LQ45 Index

AhmedLubis

2026-07-23

BAYESIAN GAME ANALYSIS ON THE LQ45 INDEX (EMPIRICAL & INTEGRATIVE)

```
# 1. LOAD REQUIRED LIBRARIES
```

```
library(rvest)
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.3
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
library(quantmod)
```

```
## Loading required package: xts
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## as.Date, as.Date.numeric
```

```
##
```

```
## ##### Warning from 'xts' package #####
```

```
## #
```

```
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
```

```
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
```

```
## # source() into this session won't work correctly. #
```

```
## #
```

```
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
```

```
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
```

```
## # dplyr from breaking base R's lag() function. #
```

```
## #
```

```
## # Code in packages is not affected. It's protected by R's namespace mechanism #
```

```
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
```

```
## #
```

```
## #####
```

```
##
```

```
## Attaching package: 'xts'
```

```
## The following objects are masked from 'package:dplyr':  
##  
## first, last
```

```
## Loading required package: TTR
```

```
## Registered S3 method overwritten by 'quantmod':  
## method from  
## as.zoo.data.frame zoo
```

```
library(zoo)  
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
library(tidyr)
```

```
# 2. AUTOMATED WEB SCRAPING: LQ45 CONSTITUENT LIST  
message("Extracting the list of LQ45 constituents from Wikipedia...")
```

```
## Extracting the list of LQ45 constituents from Wikipedia...
```

```
url <- "https://id.wikipedia.org/wiki/Indeks_LQ45"  
page <- read_html(url)  
tables <- page %>% html_table(fill = TRUE)  
lq45_raw <- tables[[1]]  
  
# Filter and format ticker symbols for Yahoo Finance (.JK extension)  
tickers <- lq45_raw[[1]] %>%  
  paste0(".JK") %>%  
  na.omit() %>%  
  unique()  
  
# Limit the number of tickers for stability during sample execution  
# (e.g., top 5 tickers)  
selected_tickers <- tickers[1:5]  
message("Tickers to be analyzed: ", paste(selected_tickers, collapse = ", "))
```

```
## Tickers to be analyzed: AADI.JK, ADMR.JK, ADRO.JK, AKRA.JK, AMMN.JK
```

```
# 3. EMPIRICAL DATA ACQUISITION (LOOP VIA YAHOO FINANCE API)  
get_stock_data <- function(ticker) {  
  tryCatch({  
    message("Downloading data for: ", ticker)  
    data <- getSymbols(ticker, src = "yahoo",  
                      from = "2023-01-01",  
                      to = "2025-12-31",  
                      auto.assign = FALSE)  
  
    df <- data.frame(  
      date = index(data),  
      price = as.numeric(Ad(data)),  
      volume = as.numeric(Vo(data)),  
      ticker = ticker  
    )  
    return(df)  
  }, error = function(e) {  
    message("Failed to download data for ticker: ", ticker)  
    return(NULL)  
  })  
}  
  
all_lq45_raw <- lapply(selected_tickers, get_stock_data) %>% bind_rows()
```

```
## Downloading data for: AADI.JK
```

```
## Downloading data for: ADMR.JK
```

```
## Downloading data for: ADRO.JK
```

```
## Downloading data for: AKRA.JK
```

```
## Downloading data for: AMMN.JK
```

```
# 4. FEATURE ENGINEERING & EMPIRICAL PROXIES (NO RANDOM COMPONENT)
message("Calculating technical indicators and strategic proxies...")
```

```
## Calculating technical indicators and strategic proxies...
```

```
# Helper function to compute RSI manually within a data frame
calculate_rsi <- function(price, n = 14) {
  delta <- c(NA, diff(price))
  gain <- ifelse(delta > 0, delta, 0)
  loss <- ifelse(delta < 0, -delta, 0)

  avg_gain <- rollmean(gain, k = n, fill = NA, align = "right")
  avg_loss <- rollmean(loss, k = n, fill = NA, align = "right")

  rs <- avg_gain / (avg_loss + 1e-10)
  rsi <- 100 - (100 / (1 + rs))
  return(rsi)
}

market_data <- all_lq45_raw %>%
  group_by(ticker) %>%
  arrange(date) %>%
  mutate(
    return = log(price / lag(price)),
    volatility = zoo::rollapply(return, width = 10,
                               FUN = sd,
                               fill = NA,
                               align = "right"),
    rsi_lag = lag(calculate_rsi(price, n = 14), 1)
  ) %>%
  na.omit() %>%
  ungroup()

# Empirical Game Variable Construction:
# Player type is determined via volume proxy
# (above median volume = Informed Dominated)
median_vol <- median(market_data$volume, na.rm = TRUE)

market_game_data <- market_data %>%
  mutate(
    prior_informed = round(volume / (volume + median_vol), 2),
    # Prior probability proxy
    player_type = ifelse(volume > median_vol, "Informed", "Uninformed"),

    # Actions determined ex-ante using day t-1 RSI momentum criteria
    action = case_when(
      rsi_lag > 60 ~ "Buy",
      rsi_lag < 40 ~ "Sell",
      TRUE ~ "Hold"
    )
  )

# Theoretical Payoff Parameter Estimation/Specifications
lambda_risk <- 0.5 # Volatility penalty
theta_info <- 0.01 # Value of information gain for the Informed group

market_game_data <- market_game_data %>%
  mutate(
    info_premium = ifelse(player_type == "Informed", theta_info, 0),
    payoff = case_when(
      action == "Buy" ~ return - (lambda_risk * volatility) + info_premium,
      action == "Sell" ~ -return - (lambda_risk * volatility),
      TRUE ~ 0 - (0.1 * lambda_risk * volatility)
    )
  )
# Small holding cost
```

```
# 5. OPTIMIZATION ENGINE: BAYESIAN NASH EQUILIBRIUM SIMULATION
message("Running Bayesian Nash Equilibrium search simulation...")
```

```
## Running Bayesian Nash Equilibrium search simulation...
```

```
# Function to search for Mixed Strategy Bayesian Nash Equilibrium (BNE)
solve_bne <- function(p_informed, r, vol) {
  # Expected Utility Definitions for Each Player Action
  u_buy <- p_informed * (r - (lambda_risk * vol) + theta_info) +
    (1 - p_informed) * (r - (lambda_risk * vol))
  u_sell <- p_informed * (-r - (lambda_risk * vol)) +
    (1 - p_informed) * (-r - (lambda_risk * vol))
  u_hold <- -0.05 * vol

  utils <- c(Buy = u_buy, Sell = u_sell, Hold = u_hold)
  bne_strategy <- names(utils)[which.max(utils)]
  return(bne_strategy)
}

market_game_data$BNE_Equilibrium <- mapply(
  solve_bne,
  market_game_data$prior_informed,
  market_game_data$return,
  market_game_data$volatility
)

print("Simulation Completed. Game Theory Panel Data Output Structure:")
```

```
## [1] "Simulation Completed. Game Theory Panel Data Output Structure:"
```

```
print(head(market_game_data[, c("date", "ticker",
                                "player_type", "action",
                                "payoff", "BNE_Equilibrium")]))
```

```
## # A tibble: 6 × 6
##   date      ticker player_type action  payoff BNE_Equilibrium
##   <date>    <chr>    <chr>    <chr>    <dbl> <chr>
## 1 2023-01-24 ADMR.JK Uninformed Hold   -0.00131 Hold
## 2 2023-01-24 ADRO.JK Informed  Sell    0.00307 Sell
## 3 2023-01-24 AKRA.JK Uninformed Hold   -0.000942 Buy
## 4 2023-01-25 ADMR.JK Uninformed Hold   -0.00131 Hold
## 5 2023-01-25 ADRO.JK Informed  Sell    0.0135 Sell
## 6 2023-01-25 AKRA.JK Uninformed Hold   -0.000916 Hold
```

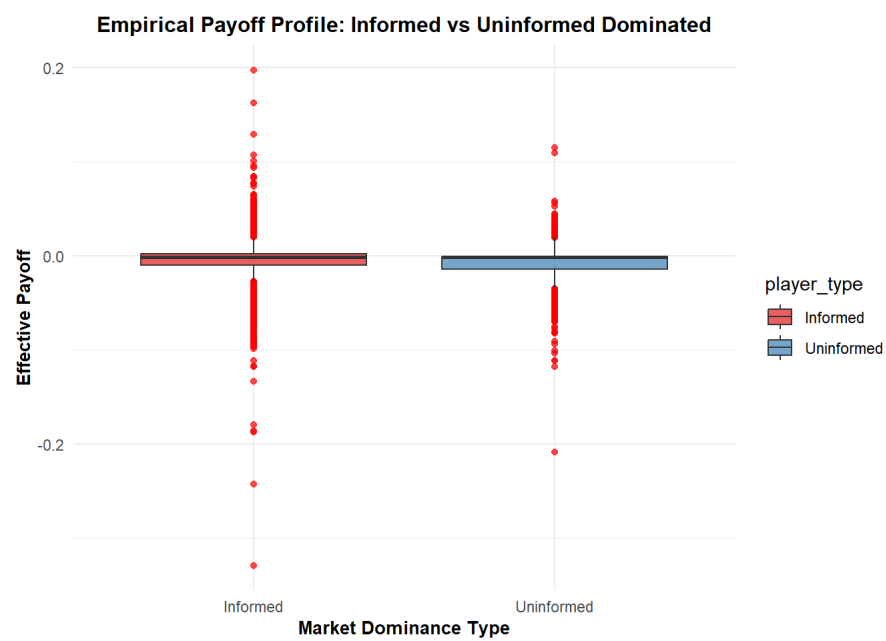
```
# 6. GRAPHICAL VISUALIZATION
thesis_theme <- theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
    axis.title = element_text(face = "bold", size = 10)
  )

# Plot 1: Empirical Payoff Distribution by Investor Type
p1 <- ggplot(market_game_data,
  aes(x = player_type, y = payoff, fill = player_type)) +
  geom_boxplot(alpha = 0.7, outlier.colour = "red") +
  labs(title = "Empirical Payoff Profile: Informed vs Uninformed Dominated",
    x = "Market Dominance Type", y = "Effective Payoff") +
  scale_fill_brewer(palette = "Set1") + thesis_theme

# Plot 2: Convergence of Actual Strategy vs Theoretical BNE Prediction
bne_comparison <- market_game_data %>%
  select(action, BNE_Equilibrium) %>%
  gather(key = "Model_Type", value = "Strategy")

p2 <- ggplot(bne_comparison, aes(x = Strategy, fill = Model_Type)) +
  geom_bar(position = "dodge", alpha = 0.8) +
  labs(title = "Strategy Distribution Comparison: Actual vs Predicted BNE",
    x = "Strategy Choice", y = "Frequency of Occurrence") +
  scale_fill_viridis_d(option = "E") + thesis_theme

print(p1)
```



```
print(p2)
```

